

Assignment 4

Each offering of a course (i.e. a section) can have many Teaching assistants;f each teaching assistant is a student. Extend the existing schema(Add/Alter tables) to accommodate this requirement.

```
→ CREATE TABLE ta ( s_id VARCHAR(5), course_id VARCHAR(20), sec_id
VARCHAR(10), semester  VARCHAR(10), year DECIMAL(4,0),
PRIMARY KEY (s_id, course_id, sec_id, semester, year),
FOREIGN KEY (s_id) REFERENCES student(ID),
FOREIGN KEY (course_id, sec_id, semester, year) REFERENCES section(course_id, sec_id,
semester, year)
);
```

```
MYSQL root@localhost (university) > describe ta;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| s_id  | varchar(5)    | NO   | PRI | <null>  |       |
| course_id | varchar(20)  | NO   | PRI | <null>  |       |
| sec_id | varchar(10)   | NO   | PRI | <null>  |       |
| semester | varchar(10)  | NO   | PRI | <null>  |       |
| year  | decimal(4,0)  | NO   | PRI | <null>  |       |
+-----+-----+-----+-----+-----+-----+

5 rows in set
Time: 0.005s
MYSQL root@localhost (university) >
[F3] Multiline: OFF
```

Alter the schema to allow a student to have multiple advisors and make sure that you are able to insert multiple advisors for a student.

→

```
ALTER TABLE advisor DROP FOREIGN KEY advisor_ibfk_1;
ALTER TABLE advisor MODIFY i_ID VARCHAR(5) NOT NULL, DROP PRIMARY KEY, ADD
PRIMARY KEY (s_ID, i_ID);
ALTER TABLE advisor ADD CONSTRAINT fk_advisor_instructor FOREIGN KEY (i_ID)
REFERENCES instructor(ID);
```

Assignment 4

Testing

INSERT INTO advisor VALUES ('00128', '10101');

insert into advisor values ('00128', '12121');

```
MySQL root@localhost (university) > select * from advisor where s_ID = 00128;
+-----+-----+
| s_ID | i_ID |
+-----+-----+
| 00128 | 10101 |
| 00128 | 12121 |
| 00128 | 45565 |
+-----+-----+

3 rows in set
Time: 0.003s
MySQL root@localhost (university) >
```

Write SQL queries on the modified schema. You will need to insert data to ensure the query results are not empty.

- Find all students who have more than 3 advisors
- Find all students who are co-advised by Prof. Srinivas and Prof. Ashok.
- Find students advised by instructors from different departments. etc.

→ INSERT INTO instructor (ID, name, dept_name, salary)
VALUES ('99999', 'Ashok', 'Elec. Eng.', 60000);

```
INSERT INTO advisor (s_ID, i_ID) VALUES
('00128', '10101'),
('00128', '12121'),
('00128', '22222'),
('00128', '15151'),
('00128', '99999'),
('12345', '10101'),
('12345', '99999'),
('23121', '12121'),
('00128', '22222'),
('23121', '22222');
```

Find all students who have more than 3 advisors

```
SELECT s.ID, s.name, COUNT(*) AS advisor_count
FROM student s
JOIN advisor a ON s.ID = a.s_ID
GROUP BY s.ID, s.name
```

Assignment 4

HAVING COUNT(*) > 3;

```
Time: 0.001s
MYSQL root@localhost (university) > SELECT s.ID, s.name, COUNT(*) AS advisor_count
-> FROM student s
-> JOIN advisor a ON s.ID = a.s_ID
-> GROUP BY s.ID, s.name
-> HAVING COUNT(*) > 3;

+-----+-----+-----+
| ID    | name  | advisor_count |
+-----+-----+-----+
| 00128 | Zhang | 4             |
+-----+-----+-----+

1 row in set
Time: 0.004s
MYSQL root@localhost (university) >
```

Find all students who are co-advised by Prof. Srinivas and Prof. Ashok

```
INSERT INTO advisor (s_ID, i_ID) VALUES ('12345', '99999');
SELECT s.ID, s.name
FROM student s
JOIN advisor a ON s.ID = a.s_ID
JOIN instructor i ON a.i_ID = i.ID
WHERE i.name IN ('Srinivasan', 'Ashok')
GROUP BY s.ID, s.name
HAVING COUNT(DISTINCT i.name) = 2;
```

```
MYSQL root@localhost (university) > INSERT INTO advisor (s_ID, i_ID)
-> VALUES
-> ('12345', '99999');

Query OK, 1 row affected
Time: 0.008s
MYSQL root@localhost (university) > SELECT s.ID, s.name
-> FROM student s
-> JOIN advisor a ON s.ID = a.s_ID
-> JOIN instructor i ON a.i_ID = i.ID
-> WHERE i.name IN ('Srinivasan', 'Ashok')
-> GROUP BY s.ID, s.name
-> HAVING COUNT(DISTINCT i.name) = 2;

+-----+-----+
| ID    | name  |
+-----+-----+
| 12345 | Shankar |
+-----+-----+

1 row in set
Time: 0.007s
MYSQL root@localhost (university) >
```

Assignment 4

Find students advised by instructors from different departments. etc.

```
SELECT s.ID, s.name
FROM student s
JOIN advisor a ON s.ID = a.s_ID
JOIN instructor i ON a.i_ID = i.ID
GROUP BY s.ID, s.name
HAVING COUNT(DISTINCT i.dept_name) > 1;
```

```
Time: 0.007s
MYSQL root@localhost (university) > SELECT s.ID, s.name
-> FROM student s
-> JOIN advisor a ON s.ID = a.s_ID
-> JOIN instructor i ON a.i_ID = i.ID
-> GROUP BY s.ID, s.name
-> HAVING COUNT(DISTINCT i.dept_name) > 1;

+-----+-----+
| ID    | name  |
+-----+-----+
| 00128 | Zhang |
| 12345 | Shankar |
+-----+-----+

2 rows in set
Time: 0.006s
MYSQL root@localhost (university) >
```

Delete all information in the database which is more than 10 years old. Add data as necessary to verify your query.

```
DELETE FROM section
WHERE year < 2016;
```

```
MYSQL root@localhost (university) > DELETE FROM section
-> WHERE year < 2016;
```

You're about to run a destructive command.

Do you want to proceed? (y/n): y

Your call!

Query OK, 15 rows affected

Time: 0.011s

```
MYSQL root@localhost (university) > SELECT *
-> FROM section
-> WHERE year < 2016;
```

```
+-----+-----+-----+-----+-----+-----+-----+
| course_id | sec_id | semester | year | building | room_number | time_slot_id |
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
```

0 rows in set

Time: 0.003s

Assignment 4

Delete the course CS 101. Any course which has CS 101 as a prereq should remove CS 101 from its prereq set. Create a cascade constraint to enforce the above rule, and verify that it is working.

```
ALTER TABLE prereq DROP FOREIGN KEY prereq_ibfk_2;
```

```
ALTER TABLE prereq ADD CONSTRAINT fk_prereq_course FOREIGN KEY  
(prereq_id) REFERENCES course(course_id) ON DELETE CASCADE;  
SHOW CREATE TABLE prereq;
```

```
DELETE FROM course  
WHERE course_id = 'CS-101';  
--verify  
SELECT *  
FROM course  
WHERE course_id = 'CS-101';
```

```
MySQL root@localhost (university) > ALTER TABLE prereq  
-> ADD CONSTRAINT fk_prereq_course  
-> FOREIGN KEY (prereq_id)  
-> REFERENCES course(course_id)  
-> ON DELETE CASCADE;  
You're about to run a destructive command.  
Do you want to proceed? (y/n): y  
Your call!  
Query OK, 7 rows affected  
Time: 0.054s  
MySQL root@localhost (university) > SHOW CREATE TABLE prereq;  
+-----+  
| Table | Create Table  
+-----+  
| prereq | CREATE TABLE `prereq` (  
|         |   `course_id` varchar(8) NOT NULL,  
|         |   `prereq_id` varchar(8) NOT NULL,  
|         |   PRIMARY KEY (`course_id`,`prereq_id`),  
|         |   KEY `fk_prereq_course` (`prereq_id`),  
|         |   CONSTRAINT `fk_prereq_course` FOREIGN KEY (`prereq_id`) REFERENCES `course` (`course_id`) ON DELETE CASCADE,  
|         |   CONSTRAINT `prereq_ibfk_1` FOREIGN KEY (`course_id`) REFERENCES `course` (`course_id`) ON DELETE CASCADE  
|         | ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci  
+-----+  
1 row in set  
Time: 0.004s  
MySQL root@localhost (university) >
```

Assignment 4

```
MYSQL root@localhost (university) > DELETE FROM course
                                -> WHERE course_id = 'CS-101';
You're about to run a destructive command.
Do you want to proceed? (y/n): y
Your call!
Query OK, 1 row affected
Time: 0.009s
MYSQL root@localhost (university) > SELECT *
                                -> FROM course
                                -> WHERE course_id = 'CS-101';
+-----+-----+-----+-----+
| course_id | title | dept_name | credits |
+-----+-----+-----+-----+
0 rows in set
Time: 0.002s
MYSQL root@localhost (university) >
```